esprit
Se former autrement

PATRON DE CONCEPTION

Cours présenté par Tarek Ayari

DESIGN PATTERN

- Un design pattern ou patron de conception est une bonne pratique en réponse à un problème de conception d'un logiciel.
- Il a pour objectif de représenter un schéma reconnu comme étant **une convention** ou à suivre, afin de répondre efficacement à un **problème récurrent**.
- Ces solutions proviennent de **l'expérience de développeurs** ayant rencontrés des problèmes de conceptions.
- Un design pattern est utilisé la majorité du temps pour **diminuer les délais de développement logiciel** afin de répondre à une question d'efficience et d'ergonomie du code.

DISPOSE PATTERN

- Dans la programmation orientée objet, le pattern « Dispose » est un modèle de conception pour la gestion des ressources mémoires.
- Dans ce modèle, une ressource est détenue par un objet, et libérée en appelant une méthode - généralement appelée **dispose** - qui libère toutes les ressources auxquelles l'objet est retenu.
- De nombreux langages proposent des mécanismes pour éviter d'avoir à appeler explicitement la méthode dispose dans des situations courantes.

DISPOSE PATTERN: LE GARBAGE COLLECTOR (GC)

- Le garbage collector (GC) de .NET est une partie du Common Language Runtime (CLR) de .NET et est disponible pour tous les langages de programmation .NET.
- Le GC a été conçu pour gérer l'allocation de mémoire, et il fait un excellent travail pour gérer la mémoire allouée directement aux **objets.NET natifs**.
- Les ressources non managées sont tout ce que le garbage collector ne peut pas gérer, par exemple : les fichiers ouverts, les connexions de base de données ...
- L'utilisation principale de ce « design pattern » est de libérer des ressources non managées. Cela réduit la dépendance à l'égard du GC et fournit aux utilisateurs un moyen de récupérer les ressources de façon déterministe.

DISPOSE PATTERN: IMPLEMENTATION

Pour implémenter le pattern Dispose, votre objet doit implémenter l'interface **System.IDisposable**:

```
public interface IDisposable
{
   void Dispose();
}
```

Afin d'utiliser l'interface IDisposable, vous devez déclarer une classe comme celle-ci :

```
public class MyClass : IDisposable
{
    public void Dispose()
    {
        // Effectuez le nettoyage de n'importe quel objet ici.

        // Si vous héritez d'une autre classe qui  implémente également IDisposable, n'oubliez pas de
        // Dispose() aussi bien.
    }
}
```

REPOSITORY PATTERN

- Ce pattern répond à un besoin d'accès aux données stockées en base. Son but principal est d'isoler la couche d'accès aux données de la couche métier.
- Il expose diverses méthodes s'appuyant sur un modèle CRUD (Create, Read, Update, Delete).
- Ce modèle fournis une centralisation de transaction aux données.
- Ils visent à créer une **couche d'abstraction** entre la **couche d'accès aux données** et la **couche logique métier** d'une application.

REPOSITORY PATTERN : CRÉATION

- Un repository est de manière générale, basé sur cinq méthodes nécessaires à l'accès aux données :

- La sélection d'un enregistrement d'une table via son ID
- La sélection de tous les enregistrements
- La mise à jour
- L'insertion
- La suppression

Cette liste est cependant loin d'être exhaustive, vous pouvez l'étendre à votre convenance afin d'implémenter des méthodes répondant à vos besoins.

REPOSITORY PATTERN : EXEMPLE

- Créer une interface générique disposant de toutes les opérations CRUD :

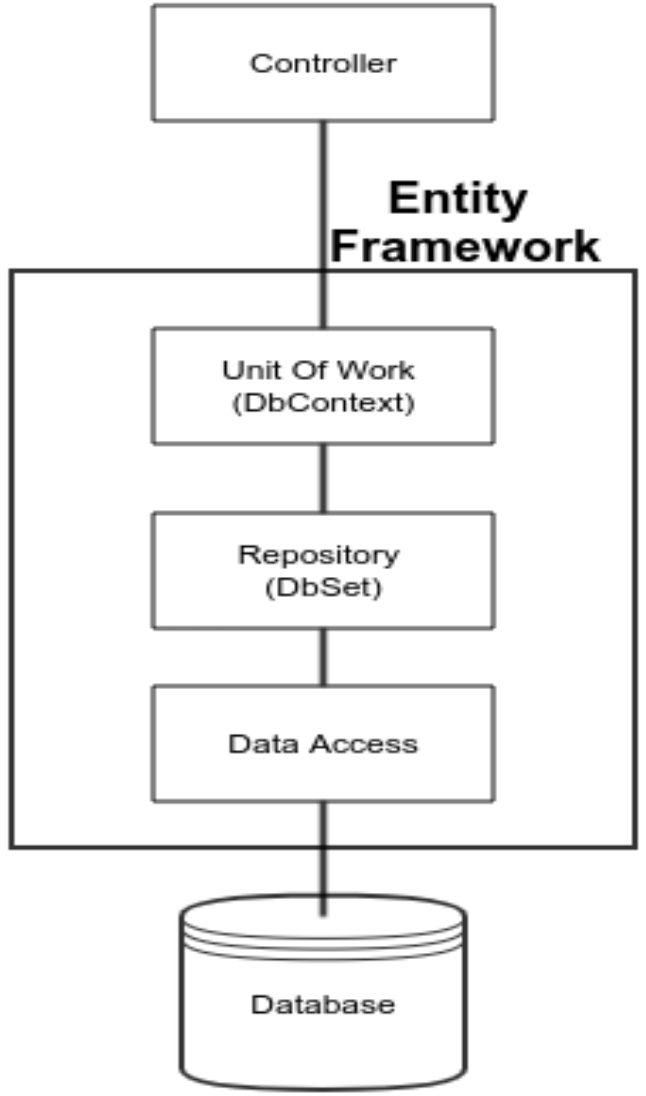
GetAll()): Cette méthode est destinée à retourner toutes les entités d'une table sous forme de collection.
GetById (): Cette méthode retourne une entité unique d'une table via sa clé primaire.
Insert(): Cette méthode prend une entité en paramètre et l'ajoute au DbSet associé.
Delete(): Cette méthode prend une entité en paramètre et la supprime du DbSet associé.
Update(): Cette méthode mis a jour une entité en paramètre du DbSet associé.
Save (): Cette méthode enregistre les modifications apportées à la base de données.

```
public interface IRepository<TEntity> where TEntity:class
  {
      IEnumerable<TEntity> GetAll();
      TEntity GetSingleById(int Id);
      void Insert(TEntity obj);
      void Update(TEntity obj);
      void Delete(int Id);
      void Save();
  }
```

UNIT OF WORK PATTERN

Le pattern Unit of work a deux tâches importantes :

- Conserver la liste de demandes d'écriture à la base de donnée au même endroit (la liste peut contenir plusieurs demandes d'insertion, de mise à jour et de suppression)
- Envoyer ces demandes d'écriture en une seule transaction à la base de données.
- Toutes les transactions telles que l'insertion / mise à jour / supprimer et ainsi de suite sont fait en une seule transaction, plutôt que de faire des transactions de bases de données multiples.



UNIT OF WORK PATTERN : EXEMPLE

```
public interface IUnitOfWork
{
        void SaveChanges();
        DbContext Context { get; }
}
```

```
public class UnitOfWork : IUnitOfWork {
        private readonly DbContext _context;
        private bool disposed = false;

        public UnitOfWork(IDbContextFactory contextFactory)
        {
            _context = contextFactory.GetContext();
        }

        public void SaveChanges()
        {
            if (_context != null)
            {
                _context.SaveChanges();
            }
        }
```